

GuardIA Dashboard — Manual Técnico Completo

Versão: 1.0.0

Data: 23 de Julho de 2026

Autor: Manus AI para Zênite Tech

Projeto: guardia-dashboard (guardia-vms.zenitetech.com)

Sumário

1. [Visão Geral da Arquitetura](#)
2. [Componentes do Sistema](#)
3. [Banco de Dados \(Supabase\)](#)
4. [Connector Python On-Prem](#)
5. [Dashboard Web \(React\)](#)
6. [Sistema de Alertas Inteligentes](#)
7. [Backup e Retenção de Dados](#)
8. [Segurança e RLS](#)
9. [Credenciais e Acessos](#)
10. [Troubleshooting](#)
11. [Próximos Passos e Roadmap](#)

1. Visão Geral da Arquitetura

O GuardIA é uma plataforma de monitoramento de segurança inteligente que integra câmeras P6S com reconhecimento facial, controle de acesso e detecção de veículos. A arquitetura segue um modelo **edge-cloud hybrid**, onde um connector on-prem coleta eventos das câmeras e os envia para o Supabase (nuvem), que por sua vez alimenta o dashboard web em tempo real via WebSocket.

```
Câmeras P6S (bancada D1-D6)
  ↓ HTTP polling (3-5s)
Connector Python (Raspberry Pi / PC on-prem)
  ↓ REST API + Storage upload
Supabase (Postgres + Realtime + Storage + Auth)
  ↓ WebSocket (realtime) + REST (queries)
GuardIA Dashboard (React 19 + Tailwind 4, browser)
```

Fluxo de dados em detalhe

1. O **Connector Python** faz polling HTTP nas câmeras P6S a cada 3-5 segundos, consultando a API local de cada câmera (`/api/event/search`)
2. Quando detecta um novo evento de reconhecimento facial, o connector envia o evento para a tabela `camera_events` no Supabase via REST API (`POST /rest/v1/camera_events`)
3. As fotos (captura + reconhecimento) são enviadas para o bucket Storage `event-images` e as URLs são armazenadas no evento
4. O **Supabase Realtime** notifica o dashboard via WebSocket sobre o novo evento INSERT
5. O **Dashboard** exibe o evento em tempo real no mosaico de câmeras, na timeline 24h, nos cards de eventos e nas notificações push
6. O motor de **alertas inteligentes** classifica o evento (crítico/warning/info) e dispara notificações com som e ações rápidas

2. Componentes do Sistema

Componente	Tecnologia	Localização	Função
Dashboard Web	React 19 + Tailwind 4 + shadcn/ui	Manus Cloud (guardia-vms.zenitetech.com)	Interface de monitoramento em tempo real
Backend/Banco	Supabase (Postgres + Auth + Storage + Realtime)	Supabase Cloud (ycqrgrczrunvyivxfnch.supabase.co)	Persistência, autenticação, storage e realtime
Connector	Python 3.11+ (requests, psycopg2)	On-prem (Raspberry Pi ou PC da bancada)	Polling das câmeras e envio de eventos
Câmeras	P6S (H5AI-50, F4C-T, T5AI)	Rede local 192.168.254.x	Captura e reconhecimento facial
Backup	Python script (backup_supabase.py)	On-prem ou cron job	Export diário de tabelas para Storage

3. Banco de Dados (Supabase)

3.1 Tabelas

camera_events

Tabela principal — recebe todos os eventos das câmeras.

Coluna	Tipo	Descrição
event_id	TEXT PK	ID único do evento (gerado pela câmera)
camera_serial	TEXT NOT NULL	Serial da câmera (ex: D2, D3)
camera_name	TEXT	Nome amigável da câmera
event_type	TEXT NOT NULL	Tipo: face, vehicle, motion, alarm, access
face_list	TEXT	Lista facial: "Stranger", "BlackList", ou nome da lista
person_name	TEXT	Nome da pessoa reconhecida (null se stranger)
face_score	INTEGER	Score de confiança do reconhecimento (0-100)
recognize_image	TEXT	URL da imagem de reconhecimento no Storage
capture_image	TEXT	URL da imagem de captura no Storage
event_time	TIMESTAMPTZ	Timestamp do evento na câmera
created_at	TIMESTAMPTZ	Timestamp de inserção no banco
attributes	JSONB	Atributos extras (direction, plate, etc.)
annotations	JSONB	Anotações feitas pelo operador no dashboard

Índices: `idx_camera_events_time` (event_time DESC), `idx_camera_events_camera` (camera_serial)

profiles

Perfis de usuário com roles de acesso.

Coluna	Tipo	Descrição
id	UUID PK	ID do usuário (FK para auth.users)
email	TEXT	E-mail do usuário
full_name	TEXT	Nome completo
role	TEXT	admin, operator, viewer
created_at	TIMESTAMPTZ	Data de criação

Trigger: `on_auth_user_created` — cria automaticamente um profile quando um usuário se cadastra no Auth.

audit_logs

Rastreabilidade de ações dos operadores.

Coluna	Tipo	Descrição
id	UUID PK	ID único
user_id	UUID	ID do usuário
action	TEXT	Ação: login, logout, annotate, export, config_change
target_type	TEXT	Tipo do alvo: event, camera, user, system
target_id	TEXT	ID do alvo
details	JSONB	Detalhes da ação
created_at	TIMESTAMPTZ	Timestamp

search_presets

Presets de busca salvos pelos operadores.

Coluna	Tipo	Descrição
id	UUID PK	ID único
name	TEXT	Nome do preset
filters	JSONB	Filtros salvos
user_id	UUID	ID do usuário que criou
created_at	TIMESTAMPTZ	Data de criação

3.2 Storage Buckets

Bucket	Público	Função
event-images	Sim	Fotos de reconhecimento e captura das câmeras
backups	Não	Backups automáticos das tabelas

3.3 Realtime

A tabela `camera_events` tem Realtime habilitado. O dashboard assina o canal `camera_events_changes` para receber notificações INSERT em tempo real via WebSocket.

4. Connector Python On-Prem

4.1 Arquitetura

O connector é um serviço Python que roda na mesma rede das câmeras. Ele faz polling periódico nas APIs HTTP de cada câmera P6S, detecta novos eventos e os envia para o Supabase.

4.2 Configuração (config/config.yaml)

```
supabase:
  url: "https://ycqrgrczrunvyivxfnch.supabase.co"
  key: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  bucket: "event-images"

cameras:
  - serial: "D1"
    ip: "192.168.254.31"
    username: "admin"
    password: "SENHA_REAL_D1"
    enabled: false
  - serial: "D2"
    ip: "192.168.254.32"
    username: "admin"
    password: "SENHA_REAL_D2"
    enabled: true
  # ... D3, D4, D5, D6

polling:
  interval_seconds: 3
  timeout_seconds: 10

image_upload: true
```

4.3 Deploy

```
# 1. Copiar ZIP para o Raspberry Pi
scp guardia-connector-deploy.zip pi@RASPBERRY_IP:~/

# 2. Descompactar e instalar
unzip guardia-connector-deploy.zip
cd connector
chmod +x install.sh
./install.sh

# 3. Editar senhas das câmeras
nano config/config.yaml

# 4. Testar
python3 src/main.py

# 5. Instalar como serviço systemd
sudo cp guardia-connector.service /etc/systemd/system/
sudo systemctl enable guardia-connector
sudo systemctl start guardia-connector

# 6. Ver logs
sudo journalctl -u guardia-connector -f
```

4.4 Comandos úteis

Comando	Função
<code>sudo systemctl start guardia-connector</code>	Iniciar serviço
<code>sudo systemctl stop guardia-connector</code>	Parar serviço
<code>sudo systemctl restart guardia-connector</code>	Reiniciar serviço
<code>sudo systemctl status guardia-connector</code>	Status do serviço
<code>sudo journalctl -u guardia-connector -f</code>	Logs em tempo real
<code>sudo journalctl -u guardia-connector --since "1h ago"</code>	Logs da última hora

5. Dashboard Web (React)

5.1 Stack

- **React 19** com Wouter (client-side routing)
- **Tailwind CSS 4** com design tokens em `index.css`
- **shadcn/ui** para componentes base
- **Supabase JS SDK** para queries e realtime
- **i18n** com suporte a PT-BR, EN e ZH

5.2 Páginas

View	Rota	Descrição
Dashboard	/dashboard	Visão geral: stats, mosaico, timeline, eventos recentes
Eventos	/events	Lista completa com busca avançada e exportação
Câmeras	/cameras	Mosaico de câmeras ao vivo e lista de dispositivos
Alertas	/alerts	Alertas de segurança e anomalias
Playback	/playback	Reprodução de gravações
Veículos	/vehicles	Biblioteca de veículos cadastrados
Config GuardIA	/settings	Status do connector e integração Supabase
Config Sistema	/system-config	Rede, sistema e armazenamento NVR
Operadores	/user-admin	Administração de usuários e roles
Auditoria	/audit-log	Rastreabilidade de ações
Automações	/automations	Regras de automação
Frequência	/frequencia	Análise de frequência de pessoas
Timeline Pessoa	/person-timeline	Histórico de uma pessoa específica
Acesso Veículos	/vehicle-access	Controle de acesso de veículos
Convite Visitante	/visitor-invite	Convites de visitantes
Alertas de Ausência	/absence-alerts	Detecção de ausência anômala
Busca Semântica	/semantic-search	Busca por semântica de IA
Resumo IA	/ai-summary	Resumo gerado por IA
Controle de Elevador	/elevator	Integração com elevadores
AI Box	/ai-box	Configuração de IA

5.3 Mapeamento de dados

O dashboard mapeia as colunas do banco para o tipo `CameraEvent` esperado pela UI:

Campo DB	Campo UI	Notas
event_id	event_id, id	ID único
camera_serial	camera_serial	Serial da câmera
event_type	operator	Tipo do evento (face → FaceReco)
person_name	payload.data.name	Nome reconhecido
face_score	payload.data.score	Score de confiança
face_list	payload.data.faceList	Lista facial
recognize_image	media_urls.recognize	URL da foto
capture_image	media_urls.capture	URL da captura
event_time	timestamp	Momento do evento
created_at	created_at	Inserção no banco

6. Sistema de Alertas Inteligentes

6.1 Motor de classificação (`critical-events.ts`)

O dashboard classifica cada evento em tempo real usando regras de negócio:

Condição	Nível	Título da Notificação
face_list = “Stranger” ou pessoa sem cadastro	Crítico	“Estranho Detectado”
face_list = “BlackList”	Crítico	“Pessoa em Lista Negra”
face_score < 50	Crítico	“Rosto Não Reconhecido”
face_score 50-69	Warning	“Match Baixo”
Movimento 22h-6h	Warning	“Movimento Fora de Horário”
Veículo sem placa	Warning	“Veículo Não Identificado”
Acesso negado	Crítico	“Acesso Negado”
Alarme disparado	Crítico	“Alarme Disparado”

6.2 Notificações em tempo real (`RealtimeNotifications.tsx`)

- **Som de alerta:** Web Audio API gera tons diferentes por severidade (duplo beep 880/660Hz para crítico, beep 660Hz para warning, chime 523Hz para info)
- **Favicon badge:** contador de notificações não lidas no ícone do browser
- **Ações rápidas:** Reconhecer, Ignorar, Escalar (conforme severidade)
- **Auto-dismiss:** 10s para não-críticos; críticos permanecem até ação manual
- **Toggle de som:** botão flutuante para ligar/desligar alertas sonoros

6.3 Deduplicação

O sistema mantém um `Set` de `event_ids` já processados para evitar notificações duplicadas. O set é limpo automaticamente quando excede 500 itens.

7. Backup e Retenção de Dados

7.1 Script de backup (`scripts/backup_supabase.py`)

O script exporta as 4 tabelas (`camera_events` , `profiles` , `audit_logs` , `search_presets`) em formato JSON e CSV, e faz upload para o bucket Storage `backups` .

7.2 Agendamento (cron)

```
# Backup diário às 03h, com cleanup de backups > 30 dias
0 3 * * * SUPABASE_KEY="eyJ..." python3 /opt/guardia/scripts/backup_supabase.py --
```

7.3 Restauração

Para restaurar um backup, baixar o arquivo do Storage e importar via `psql` :

```
# Baixar backup do Storage
curl -o backup.json "https://ycqrgrczrunvyivxfnch.supabase.co/storage/v1/object/ba

# Importar via Python
python3 -c "
import json, psycopg2
data = json.load(open('backup.json'))
conn = psycopg2.connect(host='aws-0-us-east-1.pooler.supabase.com', port=6543, ...
# ... insert logic
"
```

8. Segurança e RLS

8.1 Row Level Security (RLS)

Todas as tabelas têm RLS habilitado com as seguintes policies:

Tabela	Operação	Policy
camera_events	SELECT	Pública (qualquer usuário autenticado)
camera_events	INSERT	Pública (connector usa anon key)
camera_events	UPDATE	Apenas autenticados (anotações)
profiles	SELECT	Apenas próprio usuário ou admin
profiles	INSERT	Apenas admin
audit_logs	SELECT	Apenas autenticados
audit_logs	INSERT	Apenas autenticados
search_presets	ALL	Apenas próprio usuário

8.2 Storage Policies

Bucket	Operação	Policy
event-images	SELECT	Pública (leitura de fotos)
event-images	INSERT/UPDATE	Autenticada
backups	SELECT	Autenticada
backups	INSERT/UPDATE/DELETE	Autenticada

8.3 Recomendação de segurança

Para produção, recomenda-se:

1. Substituir a anon key pela **service_role key** no connector (bypass de RLS para insert)
2. Restringir a policy de INSERT em `camera_events` para apenas a service_role
3. Manter a anon key apenas no frontend (dashboard)

9. Credenciais e Acessos

9.1 Supabase

Item	Valor
Project URL	<code>https://ycqrgrczrunvyivxfnch.supabase.co</code>
Project Ref	<code>ycqrgrczrunvyivxfnch</code>
Anon Key (JWT)	<code>eyJhbGciOiJIUzI1NiIs...</code> (configurada no dashboard e connector)
Service Role Key	Pegar em: Supabase Dashboard → Settings → API
DB Host	<code>aws-0-us-east-1.pooler.supabase.com</code>
DB Port	<code>6543</code> (pooler)
DB User	<code>postgres.ycqrgrczrunvyivxfnch</code>
DB Pass	<code>Zenitotech2026!</code>

9.2 Usuários do Dashboard

Usuário	E-mail	Senha	Role
Admin	<code>rjll70@gmail.com</code>	<code>Zenitotech2026!</code>	Admin
Operador	<code>operador@zenitotech.com</code>	<code>Zenitotech2026!</code>	Operator
Viewer	<code>visualizador@zenitotech.com</code>	<code>Zenitotech2026!</code>	Viewer

9.3 Dashboard

Item	Valor
URL	<code>guardia-vms.zenitotech.com</code>
GitHub	Repositório <code>guardia-dashboard</code> (auto-sync)
Hosting	Manus Cloud (Autoscale)

10. Troubleshooting

10.1 Dashboard não mostra eventos

Sintoma: Dashboard carrega mas não aparecem eventos ou mostra “Nenhum evento encontrado”.

Diagnóstico:

1. Verificar se o Supabase está acessível: `curl -s https://ycqrgrczrunvyivxfnch.supabase.co/rest/v1/camera_events?limit=1 -H "apikey: eyJ..."`
2. Verificar console do browser (F12) para erros de conexão
3. Verificar se as credenciais em `supabase.ts` estão corretas
4. Verificar se a tabela `camera_events` tem registros: `SELECT count(*) FROM camera_events;`

Solução comum: As credenciais podem ter expirado ou o projeto Supabase pode estar pausado (free tier).

10.2 Timestamp mostra “há NaNd”

Sintoma: Os cards de evento mostram “há NaNd” em vez do tempo relativo.

Causa: O campo `event_time` não está sendo mapeado corretamente para `timestamp` no tipo `CameraEvent`.

Solução: Verificar se `supabase.ts` linha 78 contém: `timestamp: row.event_time || row.created_at`

10.3 Connector não envia eventos

Sintoma: Connector roda mas nenhum evento aparece no Supabase.

Diagnóstico:

1. Verificar logs: `sudo journalctl -u guardia-connector -f`
2. Verificar conectividade com as câmeras: `curl http://192.168.254.32/api/event/search`
3. Verificar conectividade com Supabase: `curl https://ycqrgrczrunvyivxfnch.supabase.co/rest/v1/ -H "apikey: eyJ..."`
4. Verificar se as senhas das câmeras estão corretas no `config.yaml`

10.4 Notificações não aparecem

Sintoma: Eventos aparecem no dashboard mas não geram notificações push.

Causa: O motor de classificação pode não estar reconhecendo o formato do evento.

Solução: Verificar se `evaluateCriticalEvent` em `critical-events.ts` está recebendo os campos corretos (`faceList` , `score` , `name`) no `payload.data`.

10.5 Realtime não funciona

Sintoma: Eventos só aparecem após refresh manual, não em tempo real.

Diagnóstico:

1. Verificar se o Supabase Realtime está habilitado para a tabela: `ALTER PUBLICATION supabase_realtime ADD TABLE camera_events;`
2. Verificar console do browser para erros de WebSocket
3. Verificar se o canal está sendo inscrito corretamente em `subscribeToNewEvents`

10.6 Login falha

Sintoma: Não consegue fazer login no dashboard.

Diagnóstico:

1. Verificar se o usuário existe no Supabase Auth: `SELECT * FROM auth.users;`
 2. Verificar se o profile existe: `SELECT * FROM profiles;`
 3. Tentar reset de senha via Supabase Dashboard → Authentication → Users
-

11. Próximos Passos e Roadmap

Curto prazo (1-2 semanas)

1. **Deploy do connector na bancada** — preencher senhas reais das câmeras D2-D6 e iniciar serviço
2. **Obter service_role key** — acessar Supabase Dashboard → Settings → API e substituir no `config.yaml`
3. **Testar com câmeras reais** — validar fluxo completo: câmera → connector → Supabase → dashboard
4. **Ativar upload de imagens** — confirmar que fotos de reconhecimento aparecem no dashboard

Médio prazo (1-2 meses)

1. **Integração com WhatsApp** — enviar alertas críticos (stranger, blacklist) via WhatsApp Business API
2. **Mapa de calor de eventos** — visualização espacial com densidade de eventos por câmera/zona
3. **Exportação de relatórios PDF** — relatórios executivos com gráficos e tabelas
4. **App mobile** — versão React Native do dashboard para acesso remoto

Longo prazo (3-6 meses)

1. **Integração ONVIF** — descoberta automática de câmeras na rede
2. **Cliente MQTT nativo** — subscrição direta de eventos de câmeras que suportam MQTT
3. **IA para análise comportamental** — detecção de anomalias baseada em padrões históricos

4. **Multi-tenant** — suporte a múltiplos clientes/condomínios com isolamento de dados

Referências

- [Supabase Documentation](#)
- [Supabase Realtime Guide](#)
- [Supabase Storage Guide](#)
- [Supabase RLS Guide](#)
- [React 19 Documentation](#)
- [Tailwind CSS 4](#)
- [shadcn/ui](#)

Documento gerado por Manus AI para Zênite Tech — 23 de Julho de 2026